

COMS3011A Test

Brendan Griffiths

Contents

1 Overview	1
2 Metrics	2
2.1 File Metrics	3
2.2 Directory Metrics	3
2.3 Repository Metrics	4
2.4 Commit set Metrics	4
2.5 Author Metrics	5
3 Rubric	5

1 Overview

Time: 2.5 Hours

Submission: URL to a public repository

Git repositories tend to be very opaque to understanding. Git does not make it easy to understand how a repo has evolved, who has had the most impact where, and what parts of the project are the most volatile.

Your project manager has asked you to build a Repo Analysis Tool (RAT) that measures specific metrics of a provided repository. You need to calculate these metrics for: each developer (called an author), each file, each directory, and the entire repository. These metrics are provided below.

The RAT should be a web-app dashboard for multiple repositories. This dashboard should be filterable by:

- A repository
- An author
- A file or directory
- Commits
 - a specified period of time
 - or a manually selected list of commits

It accepts a repository in two forms:

1. A zip file of the repo with the `.git` file or directory
2. A remote repository URL that is then deeply cloned

Not every commit will have the same author, even if they were made by the same person. To address this git provides a .mailmap to merge different email addresses, the RAT should be able to merge authors using the mailmap. If no mailmap is provided, a user should still be able to merge different authors manually.

The full list of features are:

- Repository Upload: Zip and Clone URL
- Multiple Repository Support
- Author Merging
- Metric Categories:
 - File Metrics
 - Directory Metrics
 - Repository Metrics
 - Commit Set Metrics

Note: Not all features are necessary. Please review the rubric to understand what is needed.

2 Metrics

A commit h has the following properties

- A single author $h[a]$ after author merging
- A previous commit $h[p]$
 - The initial commit has $h[p] = h_{\emptyset}$, an empty commit
- A committer date $h[\text{committer-date}]$
- $h[F]$ is the set of all files
 - **Binary files are not measured**
 - Git provides a definition and detection of binary files
- $h[D]$ is the set of all directories
- An object $o \in h[F] \cup h[D]$ is identified by its path
 - Rename detection is enabled with a threshold of 50%, so *just* renaming a file should not change its metrics.
 - Making a change and renaming an object should only have the changes impact its associated metrics.
 - These changes are attributed to its new path
 - If an object is deleted (it does not exist in h but does in $h[p]$), it should be recorded as a change in the necessary metrics (lines removed) on its path.

The set $\bar{H}$ is the set of non-merge commits reachable from a specified reference commit h_r (typically HEAD)

- A commit set H is a subset of $\bar{H}$
- The commit set H_t is the commit history from UNIX timestamp t to present

$$H_t := \{h \in \bar{H} \mid t \leq h[\text{committer-date}]\}$$

- The commit set $H_{i,j}$ is the commit history from timestamp i inclusive until timestamp j exclusive

$$H_{i,j} := \{h \in \bar{H} \mid i \leq h[\text{committer-date}] < j\}$$

- $H[F]$ is the set of all files in the repository

$$H[F] := \bigcup_{h \in H} (h[F] \cup h[p][F])$$

- $H[D]$ is the set of all directories (including the root) for the commit set.

$$H[D] := \bigcup_{h \in H} (h[D] \cup h[p][D])$$

2.1 File Metrics

- File Added Lines: The number of lines added on file f from a commit h to the previous commit

$$l_{h,f}^+$$

- File Removed Lines: The number of lines removed on file f from a commit h to the previous commit

$$l_{h,f}^-$$

- File Growth: The change in number of lines on file f from a commit h to the previous commit

$$\delta_{h,f} := l_{h,f}^+ - l_{h,f}^-$$

- File Churn: The number of changed lines on file f from a commit h to the previous commit

$$\lambda_{h,f} := l_{h,f}^+ + l_{h,f}^-$$

2.2 Directory Metrics

An *immediate* object is an object that is directly below the specified directory

```
foo/
  bar.txt    -- immediate child of foo
  baz/       -- immediate child of foo
    beef.py  -- immediate child of baz
    dead.py  -- immediate child of baz
```

A file f is in a directory d at a commit h if it is in $h[F]$ or $h[p][F]$ and is an immediate child of d . Similarly for subdirectories d' and $h[D]$, $h[p][D]$.

- Directory Added Lines: The number of added lines across all immediate subdirectories d' and files f in directory d

$$l_{h,d}^+ := \sum_{f \in d} l_{h,f}^+ + \sum_{d' \in d} l_{h,d'}^+$$

- Directory Removed Lines: The number of removed lines across all immediate subdirectories d' and files f in directory d

$$l_{h,d}^- := \sum_{f \in d} l_{h,f}^- + \sum_{d' \in d} l_{h,d'}^-$$

- Directory Growth: The net growth across all immediate subdirectories d' and files f in directory d

$$\delta_{h,d} := \sum_{f \in d} \delta_{h,f} + \sum_{d' \in d} \delta_{h,d'}$$

- Directory Churn: The churn across all immediate subdirectories d' and files f in directory d

$$\lambda_{h,d} := \sum_{f \in d} \lambda_{h,f} + \sum_{d' \in d} \lambda_{h,d'}$$

2.3 Repository Metrics

Repository metrics are directory metrics on the root of the commit tree.

2.4 Commit set Metrics

- Added lines over a commit set H in either a directory or file, $o \in H[F] \cup H[D]$

$$l_{H,o}^+ := \sum_{h \in H} l_{h,o}^+$$

- Removed lines over a commit set H in either a directory or file, $o \in H[F] \cup H[D]$

$$l_{H,o}^- := \sum_{h \in H} l_{h,o}^-$$

- Growth over a commit set H in either a directory or file, $o \in H[F] \cup H[D]$

$$\delta_{H,o} := \sum_{h \in H} \delta_{h,o}$$

- Churn over a commit set H in either a directory or file, $o \in H[F] \cup H[D]$

$$\lambda_{H,o} := \sum_{h \in H} \lambda_{h,o}$$

- Modifications: The number of commits that have at least some change on file or directory $o \in H[F] \cup H[D]$

$$\mathbb{I}^n(h, o) := \begin{cases} 1 & \text{if } \lambda_{h,o} > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$n_{H,o} := \sum_{h \in H} \mathbb{I}^n(h, o)$$

- Modification frequency over a file or directory $o \in H[F] \cup H[D]$

$$\eta_{H,o} := \begin{cases} \frac{n_{H,o}}{|H|} & \text{if } |H| \neq 0 \\ 0 & \text{otherwise} \end{cases}$$

- Churn rate over a file or directory $o \in H[F] \cup H[D]$

$$\rho_{H,o} := \begin{cases} \frac{\lambda_{H,o}}{|H|} & \text{if } |H| \neq 0 \\ 0 & \text{otherwise} \end{cases}$$

2.5 Author Metrics

We need an authorship test: $\mathbb{I}(a, h) := \begin{cases} 1 & \text{if } a=h[a] \\ 0 & \text{otherwise} \end{cases}$

- Author Modifications on a file or directory, $o \in H[F] \cup H[D]$

$$n_{H,o,a} := \sum_{h \in H} \mathbb{I}(a, h) \cdot \mathbb{I}^{n(h,o)}$$

- Author Churn on a file or directory, $o \in H[F] \cup H[D]$

$$\lambda_{H,o,a} := \sum_{h \in H} \lambda_{h,o} \cdot \mathbb{I}(a, h)$$

- Author Ownership: The fraction of churn on file or directory, $o \in H[F] \cup H[D]$ from an author a

$$\omega_{H,o,a} := \begin{cases} \frac{\lambda_{H,o,a}}{\lambda_{H,o}} & \text{if } \lambda_{H,o} \neq 0 \\ 0 & \text{otherwise} \end{cases}$$

3 Rubric

Requirements are cumulative. You can only reach a tier if the previous tier is satisfied. Each tier is judged holistically.

Metric correctness is determined against a set of test repositories. Sample metrics from each of these repos will be provided from a specific commit hash. These repos are open source.

Provided repositories:

- cJSON <https://github.com/DaveGamble/cJSON.git>
- Redis <https://github.com/redis/redis.git>
- Git <https://github.com/git/git.git>

Criteria	Weight	≤ 25%	≤ 50%	≤ 75%	≤ 100%
Requirements	50%	Implemented and correct for some categories (repo, file, directory, set, author) of metrics Either: zip file or remote URL ingestion	Implemented and correct for all metrics Both: zip file and remote URL ingestion	Implemented either: Filtering, Author Merge, Multi-repo support	Implemented all: Filtering, Author Merge, Multi-repo support
Architectural & UI Design	25%	Redundant & slow metric computation, poor visualisation of metrics	Reasonable metric computation, okay visualisation of metrics	Efficient algorithms for metric computation, good visualisation of metrics	Efficient algorithms and architecture for metric computation, inspired visualisation of metrics
Usability	25%	Poor navigation, no error handling, slow performance on small (~ 1000 commits) repositories, no QoL features	Okay navigation, minimal error handling, okay performance on small repositories, slow performance on medium (~ 10000 commits) repos, minimal to none QoL features	Good navigation, error handling, good performance on medium repositories, QoL features	Excellent navigation, good performance on large (~ 100000 commits) repos

AI Declaration: Claude Web (Opus 5.5) - reviewed